Programmer’s Guide - Power Saving

www.ingchips.com

www.ingchips.cn

service@ingchips.com

010-85160285

49 3 8 803

1009

Contents

Revision History **ix**

1 Introduction **1**

1.1 Abbreviations & Terminology 1

1.2 References 2

2 Framework **3**

2.1 Sleep Modes 4

2.2 Wake up Sources 6

2.3 Involvement of Apps 7

2.4 Shutdown 8

3 API **9**

3.1 Callback events 9

3.1.1 **PLATFORM_CB_EVT_QUERY_DEEP_SLEEP_ALLOWED** 9

3.1.2 **PLATFORM_CB_EVT_BEFORE_DEEP_SLEEP** 10

3.1.3 **PLATFORM_CB_EVT_ON_DEEP_SLEEP_WAKEUP** 10

3.1.4 **PLATFORM_CB_EVT_ON_IDLE_TASK_RESUMED** 11

3.2 Interrupts 12

3.2.1 Side Note on Priority 13

3.3 Shutdown 13

3.4 The Real-time Clock 14

3.4.1 Internal real-time RC Clock 15

3.5 Timers 16

3.6 IDLE Procedure 17

3.7 Timing 18

4 Porting **21**

4.1 API 22

4.2 Port to FreeRTOS 22

4.3 Port to RT-Thread 23

4.4 Port to Huawei LiteOS 23

4.5 Port to Azure RTOS ThreadX 24

4.6 Truly No OS 24

5 Tips on Low Power Products **27**

5.1 Clock Gating 27

5.2 Choose Best Frequency 27

5.2.1 CPU 27

5.2.2 CPU + Flash 28

5.2.3 Computational Intensive vs Hardware Timing 30

6 FAQ **31**

List of Figures

2.1 Idle Process 4

3.1 Sleep Time Calculation 19

5.1 Electric Charge Consumption 29

5.2 Electric Charge Consumption (Flash $168MHz$) 29

List of Tables

1.1 Abbreviations 1
1.2 Terminology 1

2.1 Sleep modes 4
2.2 External wake up sources of ING918 6
2.3 External wake up sources of ING916 and ING20 6

3.1 Default Priorities of Interrupts 13
3.2 Tuned Frequencies by `platform_rt_rc_auto_tune` 15
3.3 Default Timing Configurations 19
3.4 Examples for LL_DELAY_COMPENSATION of ING916 20

4.1 Relevant RT-Thread APIs 23
4.2 Relevant LiteOS APIs 24
4.3 Relevant ThreadX APIs 24

5.1 ING916 CPU Current Consumption (mA) 27
5.2 ING916 CPU Current Consumption Per MHz ($\mu A/MHz$) 28
5.3 ING916 CPU Power Consumption Per MHz ($\mu W/MHz$) 28

Revision History

Version	Note	Data
0.9	Initial	2023-02-23
0.9.1	Add Timing section	2023-02-28
0.9.2	Update 32k clock	2023-03-07
0.9.2	Update Interrupt: Priority	2023-03-09
0.9.3	Minor fixes	2023-03-17
0.9.4	Add information on wake up handler	2023-04-06
0.9.5	Update information of ING916 shutdown	2023-06-05
0.9.6	32k renamed to Real-time clock	2023-11-14
0.9.7	Add PLATFORM_CB_EVT_ON_IDLE_TASK_RESUMED	2024-05-17
1.0.0	Update porting chapter	2025-04-22
1.0.1	Update for ING20	2025-04-22
1.1	Bump version	2025-11-10
1.2	Minor updates	2026-07-21

Chapter 1

Introduction

Welcome to use *INGCHIPS* 918/916 Software Development Kit.

Power saving is important in Bluetooth LE products, especially for those using batteries. This document is a guide for developing low power products from the perspective of software.

Roughly speaking, the goal of power saving is, when there is something to do, to design the system carefully to reduce power consumption; when there is nothing to do, to put the system into special modes to reduce power consumption. The latter one is covered by the power saving Framework, while the former one is briefly discussed in Tips on Low Power System.

1.1 Abbreviations & Terminology

Table 1.1: Abbreviations

Abbreviation	Notes
BLE	Bluetooth Low Energy
CPU	Central Processing Unit
IRQ	Interrupt Request
QSPI	Quad Serial Peripheral Interface
RAM	Random Access Memory
ROM	Read Only Memory
RTOS	Real-time Operating System kernel
SDK	Software Development Kit

Table 1.2: Terminology

Terminology	Notes
Cache	A small but fast memory that stores copies of data from frequently used main memory locations

Terminology	Notes
Flash	An electronic non-volatile computer storage medium
FreeRTOS	A real-time operating system kernel
ISR	Interrupt Service Routine
KiB	1024 Bytes
Share memory	Memory in SoC that can be accessed by BLE sub-system (shared by BLE and CPU sub-systems)
System memory	Memory in SoC that can be accessed by CPU sub-system, but not BLE sub-system. It is starting from $0x20000000$.
System tick	A system timer that can be used to generate periodic interrupts or measure time
Real-time clock	The clock that keeps running in power saving modes

1.2 References

1. FreeRTOS[1]
2. Mastering the FreeRTOS™ Real Time Kernel[2]

[1] https://freertos.org

[2] https://www.freertos.org/Documentation/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf

Chapter 2

Framework

The implementation of power saving in SDK is designed to be easy to use. Generally, power saving relies on the idle process in RTOS, which is usually a process[1] having the lowest priority. It's a common practice for OS(es) to have an idle process, for example, the idle process with PID 0 in Windows and Linux.

Figure 2.1 is an overview of the idle process. Once initialized, the idle process keeps trying to put the system into sleep modes to save power. It first checks how long the system could sleep, i.e. sleep time, which is generally determined by:

- When the next BLE sub-system activity occurs;
- When the next RTOS timer expires.

Note that, peripherals, such as hardware timers, UART, are not taken into account here.

Then, the idle process makes choice on sleep modes. Following factors are considered:

- Sleep time
- App's opinion;
- Current system state.

Different sleep modes have their own hard time requirements, because it needs extra time to bring the system into sleep modes and wake it up again. App's opinion tells the idle process a subset of sleep modes which are allowed to be chosen from. Current system state including BLE sub-system is also considered.

After the most suitable sleep mode which consumes lowest power, is decided, the sleep procedure stops RTOS and takes the system into the chosen sleep mode.

After waken up, the wake up procedure performs post processing to make sure everything is normal, for example, the system tick is taken care of here. This framework knows nothing about peripherals and their configurations, and it just notifies App that the system has waken up. App could take this opportunity to configure peripherals. After everything is ready, the wake up procedure allows RTOS to continue. And the whole loop is started all over again.

[1] In embedding systems, it may also be called a *task* or *thread*.

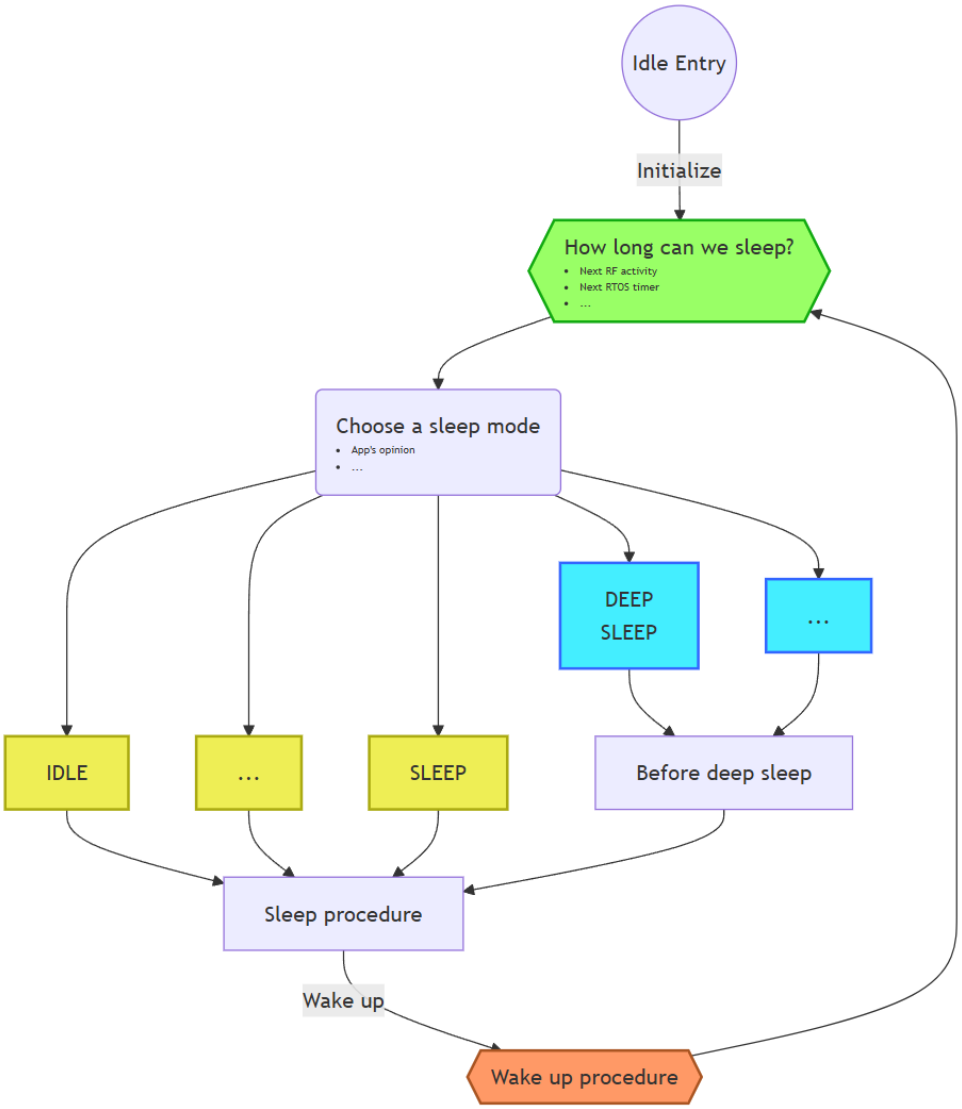

Figure 2.1: Idle Process

2.1 Sleep Modes

There are a variety of sleep modes available (supported by SoC and power saving framework) summarized in Table 2.1.

Table 2.1: Sleep modes

Mode	ING918	ING916, ING20	Notes
IDLE	✓	✓	Everything is active; CPU is waiting for interrupts.
SNOOZE	✓		Same as IDLE except RF is shutdown.
SLEEP	✓		Same as IDLE except BLE sub-system is shutdown.
DEEP SLEEP	✓	✓	CPU and peripherals are all shutdown. All memory is retained.

Mode	ING918	ING916, ING20	Notes
DEEPER SLEEP		✓	Like DEEP SLEEP. Only a small portion of memory is retained.
BLE ONLY SLEEP		✓	Like DEEP SLEEP, but BLE sub-system fully functional

IDLE mode is the slightest sleep mode, backed by the underlying CPU while other modes are defined and backed by dedicated logic in the SoC out of CPU itself. There are subtle differences between different sleep modes besides the notes in Table 2.1:

- For DEEP SLEEP on ING916 and ING92, some peripherals, such as GPIO, can be optionally put into special low power modes, rather than powered off, in which case, its output could be retained or acting as wake up sources;
- In DEEPER SLEEP, comparing with DEEP SLEEP:
  - Less peripherals are allowed to be put into special low power modes;
  - Internal $1\mu s$ timer becomes less accurate;
  - Only 16KiB of system memory is retained;
  - BLE sub-system including share memory is powered off, so all BLE configurations/parameters are lost;
- For BLE ONLY SLEEP, the system can be waken up only by BLE sub-system;
- DEEPER SLEEP is only available in the *mini* bundles.

Different sleep modes consume different amount of current, since different components in SoC consume different amount of current. Here is a very rough quantitative summary, providing a qualitative analysis:

- CPU + Flash

  Current consumption heavily depends on working frequency. It may vary from about $1mA$ to more than $10mA$.

  For ING918, CPU clock is fixed at $48MHz$, consuming $\sim 5mA$.

- RAM retention

  More memory retained, more current is needed. It costs roughly $\sim 0.02\mu A$ per KiB.

- BLE Activity

  It costs from several mA to more than $10mA$.

2.2 Wake up Sources

There are two types of sources that can cause the system to wake up:

- Internal sources

  There is only one internal source: sleep timer. It is a dedicated hardware timer used only by the idle process, configured to sleep time, and cause the system to wake up when expired.

  This timer has 32 bits, and is driven by the Real-time clock.

- External sources

  External sources are given in Table 2.2 and Table 2.3[2].

Table 2.2: External wake up sources of ING918

Mode	Sources
IDLE	All interrupts.
SNOOZE	All interrupts.
SLEEP	All interrupts.
DEEP SLEEP	EXT_INT.

Table 2.3: External wake up sources of ING916 and ING20

Mode	Sources
IDLE	All interrupts.
DEEP SLEEP	GPIO/RTC/Comparator. Configurable.
DEEPER SLEEP	GPIO. Configurable.
BLE ONLY SLEEP	BLE sub-system.

- No matter which mode SoC is in, RESET pin always works, but we don't name it as an external wake up source;
- Watchdog is an ordinary peripheral, and powered off in DEEP SLEEP, DEEPER SLEEP, and BLE ONLY SLEEP modes.

[2]Refer to *Programmer's Guide - ING916XX Peripheral* for the configuration of wake up sources

2.3 Involvement of Apps

Sleep modes can be classified into two categories, Category A that do not need the involvements of Apps (IDLE, SNOOZE, and SLEEP), and Category B that do need the involvements of Apps (DEEP SLEEP, DEEPER SLEEP, and BLE ONLY SLEEP).

This framework tried its best to minimize the involvements of Apps, and there are just three things to be handled by Apps:

1. Tell the idle process which modes in Category B are allowed to be used;

   For ING916 and ING20, Apps can take this opportunity to configure external wake up sources as well.

2. Optionally configure peripherals on-demand before entering Category B sleep modes[3];

3. (Re-)Configure peripherals after waken up from Category B sleep modes.

These three things are done in three platform event handlers respectively.

How to determine if a mode in Category B is allowed? Here are some suggestions.

- DEEP SLEEP
  - If peripherals can't be powered off now, for example, UART is running or FIFO not empty, then NO;
  - Hardware timer is used, which generates interrupts very frequently (such as $1000Hz$), then NO.
- DEEPER SLEEP
  - If DEEP SLEEP is not allowed, then NO;
  - If external wake up source does not fit the need, for example, comparator is required as a wake up source, then NO;
  - If memory outside of the retained range is in use, for example, memory allocated from Link Layer's heap, then NO;
  - If internal $1\mu s$ timer should be as accurate as in DEEP SLEEP, then NO.
- BLE ONLY SLEEP
  - If DEEP SLEEP is not allowed, then NO;
  - If other external wake up source are needed, then NO.

[3] Available from SDK v8.4.24.

2.4 Shutdown

The idle process implements an automatic and passive power saving mechanism. A proactive mechanism also exists, *Shutdown*, where developers can optionally specify a time after which the system is powered on again, and a portion of memory to be retained during shutdown. The time can be set to 0, which means to disable the power on timer, and stay in shutdown mode unless external power on signal is asserted or RESET.

- For ING918

  Shutdown is based on DEEP SLEEP, except that most of if not all memory is powered off too. As in DEEP SLEEP, the system can be powered on again by EXT_INT.

- For ING916 and ING20

  Shutdown can be backed by DEEP SLEEP (default) or DEEPER SLEEP, which can be selected by bit 0 of a debugger parameter `PLATFORM_CFG_PS_DBG_4` at present. The system can be powered on again by external sources (Table 2.3), too.

  Comparing with DEEP SLEEP, all of the system memory can be retained in DEEP SLEEP, and a bit more current is consumed.

For SDK v8.3.5 or elder versions, shutdown is always based on DEEPER SLEEP.

Chapter 3

API

There is a global switch for power saving, *Enable* or *Disable*, which can be configured by `platform_config` at any time:

```
// To enable power saving
platform_config(PLATFORM_CFG_POWER_SAVING, PLATFORM_CFG_ENABLE);

// To disable power saving
platform_config(PLATFORM_CFG_POWER_SAVING, PLATFORM_CFG_DISABLE);
```

3.1 Callback events

As discussed in Involvement of Apps, there are two events that should be handled by Apps:

3.1.1 PLATFORM_CB_EVT_QUERY_DEEP_SLEEP_ALLOWED

Handler of this event returns bit combination of allowed sleep modes in Category B. A value of 0 means that no mode in Category B is allowed at present.

```
#define PLATFORM_ALLOW_DEEP_SLEEP      ...
#define PLATFORM_ALLOW_DEEPER_SLEEP    ...
#define PLATFORM_ALLOW_BLE_ONLY_SLEEP  ...
```

When DEEPER SLEEP is allowed, DEEP SLEEP is also allowed implicitly. For example, if both DEEP and DEEPER SLEEP are allowed, then:

```
return PLATFORM_ALLOW_DEEP_SLEEP + PLATFORM_ALLOW_DEEPER_SLEEP;
```

or, simply (although not recommended),

```
return PLATFORM_ALLOW_DEEPER_SLEEP;
```

For ING916 and ING20, Apps can also configure external wake up sources in the handle.

When a sleep mode returned as allowed but is not supported in the system, the sleep mode is ignored.

3.1.2 PLATFORM_CB_EVT_BEFORE_DEEP_SLEEP[1]

When platform decides to enter one of Category B sleep modes, this event is emitted. Apps can take this opportunity to configure peripherals on-demand. The callback function shall be simple and return as soon as possible.

Input parameter `void *data` is casted from `platform_sleep_category_b_t`, representing the selected sleep mode.

For ING916 and ING20, Apps can also configure external wake up sources in the handle.

3.1.3 PLATFORM_CB_EVT_ON_DEEP_SLEEP_WAKEUP

In the handler of this event, Apps can re-initialize peripherals and do other jobs according to the needs.

Input parameter `void *data` is casted from `platform_wakeup_call_reason_t`:

```
typedef enum
{
    PLATFORM_WAKEUP_REASON_NORMAL = ...,
    PLATFORM_WAKEUP_REASON_ABORTED = ...,
} platform_wakeup_call_reason_t;
```

PLATFORM_WAKEUP_REASON_NORMAL means that this event is emitted for a normal (successful) wake up procedure, i.e. the sleep procedure is completed successfully; *PLATFORM_WAKEUP_REASON_ABORTED* means that the previous sleep procedure is aborted, i.e. sleep had not happened at all. For example, trying to go to DEEP SLEEP while EXT_INT is asserted will be a failure, and sleep will not happen.

[1] Available from SDK v8.4.24.

Event with *PLATFORM_WAKEUP_REASON_NORMAL* is always emitted; while event with *PLATFORM_WAKEUP_REASON_ABORTED* is emitted when `PLATFORM_CFG_ALWAYS_CALL_WAKEUP` is *Enabled*. For ING918, `PLATFORM_CFG_ALWAYS_CALL_WAKEUP` is default to *Disabled*; while for ING916 and ING20, default to *Enabled*. This is because for ING916 and ING20, external wake up sources such GPIO might need to be reconfigured if sleep procedure is aborted. Without this event, Apps will not be able to do such reconfiguration. Since external sources are not configurable in the case of ING918, so it is default to *Disabled*.

If `PLATFORM_CFG_ALWAYS_CALL_WAKEUP` is *Enabled*, then there will be a *PLATFORM_CB_EVT_ON_DEEP* event *definitely* for each *PLATFORM_CB_EVT_QUERY_DEEP_SLEEP_ALLOWED* event that returns a non-0 value.

At present, this handler is invoked in a *strange* context. It is in neither an ordinary process, nor an interrupt service routine. In the case of FreeRTOS, neither normal APIs or their *FromISR* variants are allowed, with only a few exceptions, such as:

- Use `xSemaphoreGiveFromISR` on a binary semaphore.

Some platform APIs might be unavailable, either. For example, `platform_get_us_time()` on ING918 would not get reliable value, while on ING916, it would hang.

3.1.4 PLATFORM_CB_EVT_ON_IDLE_TASK_RESUMED[2]

After event *PLATFORM_CB_EVT_ON_DEEP_SLEEP_WAKEUP* with reason *PLATFORM_WAKEUP_REASO* and the idle process is fully resumed from power saving, this event is emitted.

Input parameter `void *data` is always 0.

Comparing with *PLATFORM_CB_EVT_ON_DEEP_SLEEP_WAKEUP*, in this event:

- All OS functionalities are resumed;

  For NoOS variants, the callback is invoked by `platform_os_idle_resumed_hook()`. Therefore, `platform_os_idle_resumed_hook()` must be properly used.

- All platform APIs are functional;

- Event callback function is invoked in the idle process.

The prototype of event handles should be compatible of:

[2] Available from SDK v8.4.16.

```
typedef uint32_t (*f_platform_evt_cb)(void *data, void *user_data);
```

Handlers are registered by:

```
void platform_set_evt_callback(
  // the event
  platform_evt_callback_type_t type,
  // the callback function
  f_platform_evt_cb f,
  // user data that will be passed into callback function `f`
  void *user_data
);
```

3.2 Interrupts

Since CPU is powered off in Category B sleep modes, after waken up, from CPU's point of view, interrupts should be re-enabled.

For example, an UART port is used the App with both Rx and Tx interrupts are enabled. Then, in the waken up event handle, not only the peripheral itself should be re-initialized, its Rx and Tx interrupts enabled, but also the corresponding position in interrupt vector table. In the case of ARM Cortex-M processors, it is called NVIC[3].

If the ISR is registered by `platform_set_irq_callback`, then there are two options to re-enable the interrupt:

1. Call `platform_set_irq_callback` again, since this API will enable the interrupt;
2. Use `platform_enable_irq`:

```
void platform_enable_irq(
  // corresponding interrupt request type
  platform_irq_callback_type_t type,
  // flag = 1 for enable; flag = 0 for disable
  uint8_t flag);
```

If the ISR is in the table registered by `platform_set_irq_callback_table`, then use `platform_enable_irq`, too.

[3]Nested Vectored Interrupt Controller

In the case of ARM Cortex-M processors, the priority of an interrupt is identified by an unsigned integer whose width is `__NVIC_PRIO_BITS`, where a higher value represents a lower priority. Therefore, the highest priority is 0 and the lowest is (1 « __NVIC_PRIO_BITS) - 1. Note that, 0 is not allowed here[4], so the highest priority shall be 1. When using FreeRTOS, the value for highest priority is defined by and can be got from `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY`.

Priorities of interrupts are all reset to default configurations as well after waken up. Default configurations are given in Table 3.1.

Table 3.1: Default Priorities of Interrupts

Interrupt	ING918	Others
BLE sub-system	3	6
Others	5	11

3.2.1 Side Note on Priority

The BLE stack in ING91X are designed to be soft real-time as much as possible, for example, the whole system can be stopped and resumed later without lost of BLE connection if the supervision timer is not reached. For hard real-time, there are deadlines that must be met each and every time. In such cases, developers might need to change those priorities and will assign the highest priorities to those interrupts that look like to the most important. However, that is not the optimal algorithm. Rate Monotonic Algorithm[5] is a classic algorithm that should be considered firstly.

To modify priority, call `platform_read_info` to get the underlying IRQ number, then use corresponding API of CPU to modify it. For example, change the priority of UART0 to `7`, in the case of ARM Cortex-M processors:

```
IRQn_Type n = (IRQn_Type)platform_read_info(
  PLATFOFM_INFO_IRQ_NUMBER + PLATFORM_CB_IRQ_UART0);
NVIC_SetPriority(n, 7);
```

When assigning a higher priority than BLE sub-system, Link Layer APIs are **NOT** allowed to be invoked in these ISR(s) even though they are marked as *thread safe*.

3.3 Shutdown

Shutdown is initiated by calling `platform_shutdown`:

[4]This is required by software bundles.
[5]Liu and Leyland, 1973, *Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment*.

```
void platform_shutdown(
  // Duration before power on again (measured in cycles of real-time clock)
  const uint32_t duration_cycles,
  // Pointer to the start of data to be retained
  const void *p_retention_data,
  // Size of the data to be retained
  const uint32_t data_size);
```

When `duration_cycles` is zero, power on when external wake up source is asserted. When `duration_cycles` is not zero, it must be larger than a minimum value of 825 cycles (about $25.18ms$) reserved for hardware. `data_size` can be zero too if no data is needed to be retained. Only part of SYS memory starting from 0x2000000 can be retained.

If this function fails, it will return. If this function successes, CPU is powered off, so it will not return.

3.4 The Real-time Clock

The Real-time clock is crucial for power saving. There are two sources for this clock, one is internal real-time RC clock, and the other is the real-time crystal oscillator which needs an external crystal. So, platform provides APIs to select clock source, and change settings:

- **PLATFORM_CFG_RT_RC_EN**

  Enable/Disable the internal real-time RC clock. Default: Enabled.

- **PLATFORM_CFG_RT_OSC_EN**

  Enable/Disable the real-time crystal oscillator. Default: Enable.

- **PLATFORM_CFG_RT_CLK**

  Real-time clock selection. Flag is `platform_rt_clk_src_t` with default value `PLATFORM_RT_RC`:

```
typedef enum
{
  // external real-time crystal oscillator
  PLATFORM_RT_OSC,
  // internal real-time RC clock
  PLATFORM_RT_RC
} platform_rt_clk_src_t;
```

When modifying this configuration, both RT_RC and RT_OSC should be **enabled**. For ING918: both clocks must be running; To ensure a clock is running:

  - **RT_OSC**: wait until status of RT_OSC is OK;
  - **RT_RC**: wait $100\mu s$ after enabled.

  It's recommended to wait another $100\mu s$ before disabling the unused one.

- **PLATFORM_CFG_RT_CLK_ACC**

  Configure Real-time clock accuracy in *ppm*.

- **PLATFORM_CFG_RT_CLK_CALI_PERIOD**

  Real-time clock auto-calibration[6] period in seconds. Default: $3600 \times 2 = 2(hours)$.

  Beside auto-calibration, it can also be started manually by:

```
uint32_t platform_calibrate_rt_clk(void);
```

The frequency of Real-time clock can be calculated from calibration value:

$$f = \frac{65536000000}{Cali} Hz,$$

where, the calibration value $(Cali)$ is an unsigned integer returned by `platform_read_info(PLATFORM_INF`

3.4.1 Internal real-time RC Clock

The internal real-time RC clock is less accurate real-time crystal oscillator generally. It can be tuned by selecting property capacitor and resistor. This process is done by invoking:

```
uint16_t platform_rt_rc_auto_tune(void);
```

The frequency is tuned to different frequencies by `platform_rt_rc_auto_tune` at present (Table 3.2).

Table 3.2: Tuned Frequencies by `platform_rt_rc_auto_tune`

Family	Frequency (Hz)
ING918	50000
Others	32768

This process takes hundreds of milliseconds, so it recommended to use it in a thread but not in `app_main`. This process can be done only once after starting up. It also returns a value representing the best choice which can be saved and used to tune the clock directly in latter restarts:

[6] *Calibration* means to measure the frequency of this clock with another high frequency and high precision clock.

```
void platform_rt_rc_tune(
  uint16_t value  // returned by `platform_rt_rc_auto_tune`
);
```

This framework does not depends on a specific frequency, and it is possible to tune to a another frequency by calling `platform_rt_rc_auto_tune2`:

```
uint16_t platform_rt_rc_auto_tune2(
  // target frequency in Hz
  uint32_t target_frequency);
```

ING918: RC clock's power supply after waken up from DEEP SLEEP is different from starting up. Therefore, calling `platform_rt_rc_auto_tune()` after DEEP SLEEP can achieve better accuracy.

3.5 Timers

Platform also provides a few timer APIs coupled with power saving.

There is an internal counter increased by 1 per $1\mu s$, which wraps after $7953.64 days (\approx 21.79 years)$. Therefore, this timer can be treated as never wrapping practically. This counter is carefully recovered during waking up[7]. It is reset to zero when power up again from shutdown. Use `platform_get_us_time` to read this counter:

```
uint64_t platform_get_us_time(void);
```

System tick is disabled and restarted when entering and leaving sleep modes. It's difficult to properly reconfigure it accurately. Platform provides a configure item `PLATFORM_CFG_RTOS_ENH_TICK`. When enabled, accuracy of system ticks is improved but less power efficiency.

Platform also provides another timer that survives in all sleep modes and is more accurate than system ticks. Such timers can be created by calling `platform_set_timer`:

[7] For DEEPER SLEEP mode of ING916 and ING20, it is coarsely recovered comparing to other modes.

```
void platform_set_timer(
  // callback function when timer expires
  void (* callback)(void),
  // timer expires after this delay from now
  // Unit: 625us
  uint32_t delay);
```

This timer has some unique features:

- Comparing to hardware timers, this timer can be thought as "running" during power saving mode;
- Comparing to RTOS software timers, this timer is software + hardware too;
- Comparing to RTOS software timers, this timer may be more accurate in some circumstance;
- This function always succeed, and only fails when running out of memory.

`callback` is also the identifier of the timer. So below two lines defines only one timer expiring after 200 units but not two separate timers:

```
platform_set_timer(f, 100);
platform_set_timer(f, 200);
```

The callback function is called in a RTOS task (if existing), but not an ISR.

Range of `delay` is $0 \sim$ *0x7fffffff*. When `delay` is zero, the timer associated with the callback function is cleared. Since `delay` is in $625\mu s$, the maximum delay is $372.827 hours (\approx 15.5345 days)$.

3.6 IDLE Procedure

As mentioned in Sleep Modes, IDLE mode is base on CPU's feature solely. It's common for CPU to have an instruction that instruct CPU to stop and enter a slight sleep mode. It can wake up (continue to execute) *quickly* when there is any interrupt/exception, or other signals. For x86 processors, the instruction is *HLT* (halt), while for ARM Cortex-M processors, it is *wfi* (wait for interrupts) or *wfe* (wait for events). When there is only one ARM Cortex-M processor, the IDLE procedure is simply a *wfi* guarded by a pair of synchronization barriers:

```
__DSB();
__wfi();
__ISB();
```

When there is a PLL (such as ING916 and ING20), CPU and other components may run at a range of frequencies, event `PLATFORM_CB_EVT_IDLE_PROC` is defined to provide flexibility of customizing the IDLE procedure. This event is emitted each time IDLE mode is entered, and default IDLE procedure is replaced by the callback function. For example, Apps can lower clock frequencies to enjoy much low power consumption in IDLE mode if the slower response in waking up is acceptable:

```
static uint32_t idle_proc(void *dummy, void *user_data)
{
    SYSCTRL_ConfigPLLClk(DIV_PRE, LOOP, 63);
    __DSB();
    __WFI();
    __ISB();
    SYSCTRL_ConfigPLLClk(DIV_PRE, LOOP, DIV_OUTPUT);
    return 0;
}

platform_set_evt_callback(PLATFORM_CB_EVT_IDLE_PROC,
    idle_proc, NULL);
```

3.7 Timing

Finally, there are several configuration to share some information about *outside* world to the power saving framework.

- **PLATFORM_CFG_xx_TIME_REDUCTION** which helps to answer how much time the sleep can take;

  As shown in Figure 3.1, suppose now is T_0, and next action occurs at T_1. Obviously, the system can not sleep for $(T_1 - T_0)$, since there are pre-processing and post-processing in both hardware and software which will take some time too. These events callbacks are part of "pre-processing":

  - `PLATFORM_CB_EVT_QUERY_DEEP_SLEEP_ALLOWED`,
  - `PLATFORM_CB_EVT_BEFORE_DEEP_SLEEP`.

  And these events callbacks are part of "post-processing":

  - `PLATFORM_CB_EVT_ON_DEEP_SLEEP_WAKEUP`,
  - `PLATFORM_CB_EVT_LLE_INIT` (Always for ING918/ING916, and sometimes for ING20[8]).

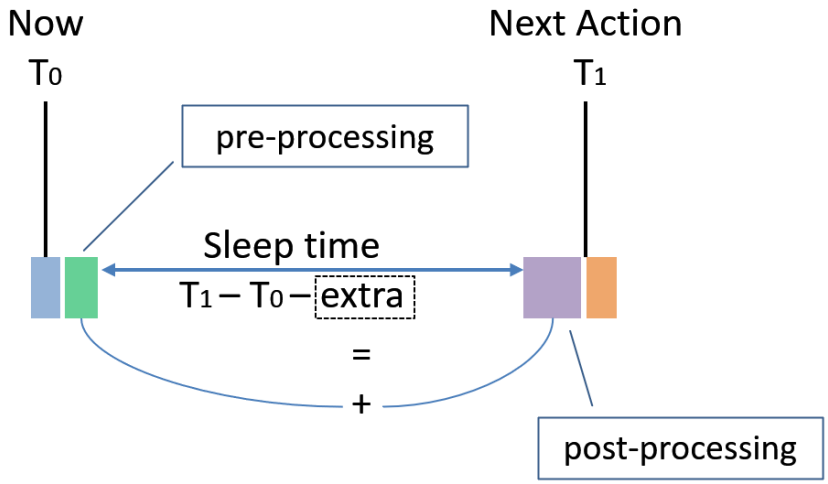

Figure 3.1: Sleep Time Calculation

Therefore extra time needs to be reduced from $(T_1 - T_0)$. The amount of time can be configured through `PLATFORM_CFG_SLEEP_TIME_REDUCTION` for SLEEP mode, and `PLATFORM_CFG_DEEP_SLEEP_TIME_REDUCTION` for DEEP SLEEP and DEEPER SLEEP modes.

`PLATFORM_CFG_SLEEP_TIME_REDUCTION` is only applicable to ING918.

- **PLATFORM_CFG_LL_DELAY_COMPENSATION** which helps to answer when starting to configure BLE sub-system.

  Suppose a BLE activity occurs at T_1 exactly, Link Layer software need to properly configure it before T_1. Since CPU may run at a variant of frequencies, time needed to do this job also varies. This configurations tells the framework how much time shall be reserved for Link Layer software: let t be PLATFORM_CFG_LL_DELAY_COMPENSATION, then Link Layer will start at $(T_1 - t)$.

  This configuration is not applicable to ING918.

Default values are given in Table 3.4.

Table 3.3: Default Timing Configurations

Item	Default for ING918 (μs)	Default for ING916 (μs)
SLEEP_TIME_REDUCTION	450	–
DEEP_SLEEP_TIME_REDUCTION	550	4000
LL_DELAY_COMPENSATION	–	200

[8]ING20 initializes LLE on demand: If a BLE activity is expected to be carried out at T_1, `PLATFORM_CB_EVT_LLE_INIT` will be part of "post-processing"; otherwise, LLE won't be initialized.

Generally, these values do not need to be updated in the case of ING918. If the wake up handler does a lot of processing, then those two reductions should be increased accordingly.

It's much more complicated for ING916 or ING20. Good values can be found by trial and error, when precise measurement is impossible. When values are too small, BLE functions will be affected, so it can be used as an indicator. Table 3.4 lists reference values for LL_DELAY_COMPENSATION of ING916.

Table 3.4: Examples for LL_DELAY_COMPENSATION of ING916

CPU Frequency (MHz)	Value (μs)
48	1200
32	2000
24	2500
16	5000
8	9000

Chapter 4

Porting

When using NoOS variants of platform bundles, power saving framework needs to be ported to other RTOS(es). The main work is to port the idle process to the targeting RTOS. The idle process[1] looks like this:

```
void idle_task(void)
{
    for (;;)
    {
        timeout_tick = rtos_next_busy_tick();
        if (platform_pre_suppress_cycles_and_sleep_processing(
          timeout_tick * cycles_per_tick) > some_limit)
        {
            rtos_stop_scheduler();
            platform_pre_sleep_processing();
            platform_post_sleep_processing();
            tick_cnt = how_long_has_I_slept();
            rtos_compensate_system_ticks(tick_cnt);
            restart_system_ticks();
            rtos_resume_scheduler();
        }
        platform_os_idle_resumed_hook();
    }
}
```

What needs to be ported is exactly those `rtos_...` functionalities:

- `rtos_next_busy_tick` returns how many ticks the CPU can be put in sleep modes;
- `rtos_stop_scheduler` stops the scheduler

[1] Since system tick interrupts are disabled during sleep, it is called tick-less low power feature in some RTOS(es).

- `rtos_compensate_system_tick` compensates ticks counter of RTOS
- `rtos_resume_scheduler` resumes the scheduler

`how_long_has_I_slept` calculates how long the last sleep was in ticks which is used to compensates ticks counter of RTOS. The result can be deduced by check system tick registers of CPU, or use other real-time timer including `platform_get_us_time`. `restart_system_tick` restarts system ticks.

Besides, optionally, similar function of `PLATFORM_CFG_RTOS_ENH_TICK` needs to be ported too.

The idle process must be the **only** process having the lowest priority. Some RTOS(es) supported multiple processes have the same lowest priority. Developers should ensure that no other processes are having the lowest priority.

4.1 API

NoOS variants of platform bundles provide a collection of APIs to cooperate the porting [2].

```
// Pre-suppress cycles and sleep processing
// @return  adjusted cycles to sleep
uint32_t platform_pre_suppress_cycles_and_sleep_processing(
  // expected cycles to sleep
  uint32_t expected_cycles
);

// Preprocessing for tick-less sleep
void platform_pre_sleep_processing(void);

// Postprocessing for tick-less sleep
void platform_post_sleep_processing(void);

// Hook for idle task got resumed
void platform_os_idle_resumed_hook(void);
```

4.2 Port to FreeRTOS

FreeRTOS[3] is a real-time operating system for micro controllers and small microprocessors. It is a cross-platform RTOS kernel that is distributed freely under the MIT open source license.

[2] `platform_pre_suppress_cycles_and_sleep_processing` is introduced in SDK v8.5.0. For elder SDK versions, use `platform_pre_suppress_ticks_and_sleep_processing` instead.

[3] https://www.freertos.org/

Platform binaries that has a built-in RTOS bundles FreeRTOS. NoOS variants are certainly portable to FreeRTOS. The portable can be done by just defining several macros to feed above APIs into FreeRTOS, such as:

```
#define SYS_CLOCK_CYCLES_PER_TICK   ( configSYSTICK_CLOCK_HZ / configTICK_RATE_HZ )

#define configPRE_SUPPRESS_TICKS_AND_SLEEP_PROCESSING(xExpectedIdleTime)  \
  do {                                                                   \
    xExpectedIdleTime =                                                  \
      platform_pre_suppress_cycles_and_sleep_processing(xExpectedIdleTime \
          * SYS_CLOCK_CYCLES_PER_TICK) / SYS_CLOCK_CYCLES_PER_TICK;      \
  } while (0)
```

A complete example can be found in SDK repository[4].

4.3 Port to RT-Thread

RT-Thread[5] is an open source embedded real-time operating system for IoT devices. It has a small size, rich features, high performance and scalability.

`rtos_...` functionalities can be mapped to relevant RT-Thread APIs as shown in Table 4.1.

Table 4.1: Relevant RT-Thread APIs

`rtos_...` Functionalities	RT-Thread APIs
`rtos_next_busy_tick`	`rt_timer_next_timeout_tick`
`rtos_stop_scheduler`	`rt_enter_critical`, `rt_hw_interrupt_disable`
`rtos_compensate_system_tick`	`rt_tick_set`
`rtos_resume_scheduler`	`rt_exit_critical`, `rt_hw_interrupt_enable`

A complete example can be found in SDK repository[6].

4.4 Port to Huawei LiteOS

Huawei LiteOS[7] was a lightweight real-time operating system for IoT devices developed by Huawei. It was open source, POSIX compliant, and has been incorporated into HarmonyOS.

[4] https://github.com/ingchips/ING918XX_SDK_SOURCE/tree/master/examples/peripheral_console_freertos
[5] https://www.rt-thread.io/
[6] https://github.com/ingchips/ING918XX_SDK_SOURCE/tree/master/examples/peripheral_console_rt-thread
[7] https://gitee.com/LiteOS/LiteOS

`rtos_...` functionalities can be mapped to relevant LiteOS APIs as shown in Table 4.2.

Table 4.2: Relevant LiteOS APIs

`rtos_...` Functionalities	LiteOS APIs
`rtos_next_busy_tick`	`OsSleepTicksGet`
`rtos_stop_scheduler`	`LOS_IntLock`
`rtos_compensate_system_tick`	`OsSysTimeUpdate`
`rtos_resume_scheduler`	`LOS_IntRestore`

A complete example can be found in SDK repository[8].

4.5 Port to Azure RTOS ThreadX

Azure RTOS ThreadX[9] is Microsoft's Real-Time Operating System (RTOS) for deeply embedded, real-time, and IoT applications. It has a small footprint, fast execution speed, and advanced features such as preemption-threshold scheduling, event chaining.

`rtos_...` functionalities can be mapped to relevant ThreadX APIs as shown in Table 4.3.

Table 4.3: Relevant ThreadX APIs

`rtos_...` Functionalities	ThreadX APIs
`rtos_next_busy_tick`	`tx_timer_get_next`
`rtos_stop_scheduler`	`TX_DISABLE`
`rtos_compensate_system_tick`	`tx_time_increment`
`rtos_resume_scheduler`	`TX_RESTORE`

A detailed explanation can be found online[10].

4.6 Truly No OS

It's possible to run NoOS bundles without any RTOS, including full functional power saving. A complete example can be found in SDK repository[11].

In the example, because there are no software timers, and no next busy tick, the whole idle process couldn't be simpler:

[8] https://github.com/ingchips/ING918XX_SDK_SOURCE/tree/master/examples-gcc/peripheral_console_liteos
[9] http://docs.microsoft.com/azure/rtos/threadx
[10] https://ingchips.github.io/blog/2022-03-11-threadx-porting/
[11] https://github.com/ingchips/ING918XX_SDK_SOURCE/tree/master/examples-gcc/peripheral_console_realtime

```
static void idle_process(void)
{
    uint32_t cycles =
      platform_pre_suppress_cycles_and_sleep_processing((uint32_t)-1);
    if (cycles < 150) return;
    enter_critical();
    platform_pre_sleep_processing();
    platform_post_sleep_processing();
    leave_critical();
    platform_os_idle_resumed_hook();
}
```

Chapter 5

Tips on Low Power Products

Here are some tips for building low power products.

5.1 Clock Gating

Make sure that clock is gated for those peripherals that are not used. Since clock of all peripherals (except SYSCTRL and the default UART[1]) are defaults to gated, so, make sure do not release the gating for those peripherals that are not used. If default UART is not used, then gate its clock.

5.2 Choose Best Frequency

For SoC that has complex clock configurations like ING916, clock frequencies of each peripheral as well as CPU itself, should be chosen with care.

5.2.1 CPU

The lower frequency, the less current is consumed. But, with a lower frequency, processing time becomes longer. Do not only measure current, but also execution time.

Table 5.1 shows current consumption[2] of ING916's CPU running at varies frequencies under room temperature.

Table 5.1: ING916 CPU Current Consumption (mA)

Frequency (MHz)	$V_{bat} = 3.3V$	$V_{bat} = 2.5V$	$V_{bat} = 1.8V$
112	8.95	11.32	14.26
64	5.97	7.17	9.26

[1]It is UART0 if there are multiple UART peripherals.
[2]Refer to *ING916X BLE5.3 SoC Test Report*.

Frequency (MHz)	$V_{bat} = 3.3V$	$V_{bat} = 2.5V$	$V_{bat} = 1.8V$
24	1.71	2.06	2.69
8	0.90	1.09	1.47

Better ways to measure power efficiency is to measure current per MHz (Table 5.2) or power per MHz (Table 5.3).

Table 5.2: ING916 CPU Current Consumption Per MHz ($\mu A/MHz$)

Frequency (MHz)	$V_{bat} = 3.3V$	$V_{bat} = 2.5V$	$V_{bat} = 1.8V$
112	89.9	101.1	127.3
64	93.3	112.0	144.7
24	71.3	85.8	112.1
8	112.6	136.3	183.8

Table 5.3: ING916 CPU Power Consumption Per MHz ($\mu W/MHz$)

Frequency (MHz)	$V_{bat} = 3.3V$	$V_{bat} = 2.5V$	$V_{bat} = 1.8V$
112	263.7	252.7	229.2
64	307.8	280.1	260.4
24	235.1	214.6	201.8
8	371.3	340.6	330.8

From these numbers we can see that:

- It's not power efficient to do computation at a very low frequency, for example, under the same condition, running at $8MHz$ consumes at least 40% more power than $112MHz$;
- $24MHz$ looks like a sweet spot if the processing delay is acceptable;
- If frequency is fixed, always try to use lower V_{bat} voltage.

5.2.2 CPU + Flash

Flash embedded in ING916 and ING20 SoC interfacing CPU through QSPI, which might become a bottleneck without cache. Since cache is shutdown in Category B sleep modes, in a short duration after waking up, the system works like without cache. Therefore, it is important to measure current consumption in such cases.

Take ING916 as an example, create a function filled with 8192 NOP(s)(no operation) that is larger than I-Cache, loop the function for 200 times, and measure the used electric charge. Figure 5.1 shows the result. It can be found that when lowering Flash frequency, much more electric charge is used. To put it simple, run Flash as fast as possible to save power.

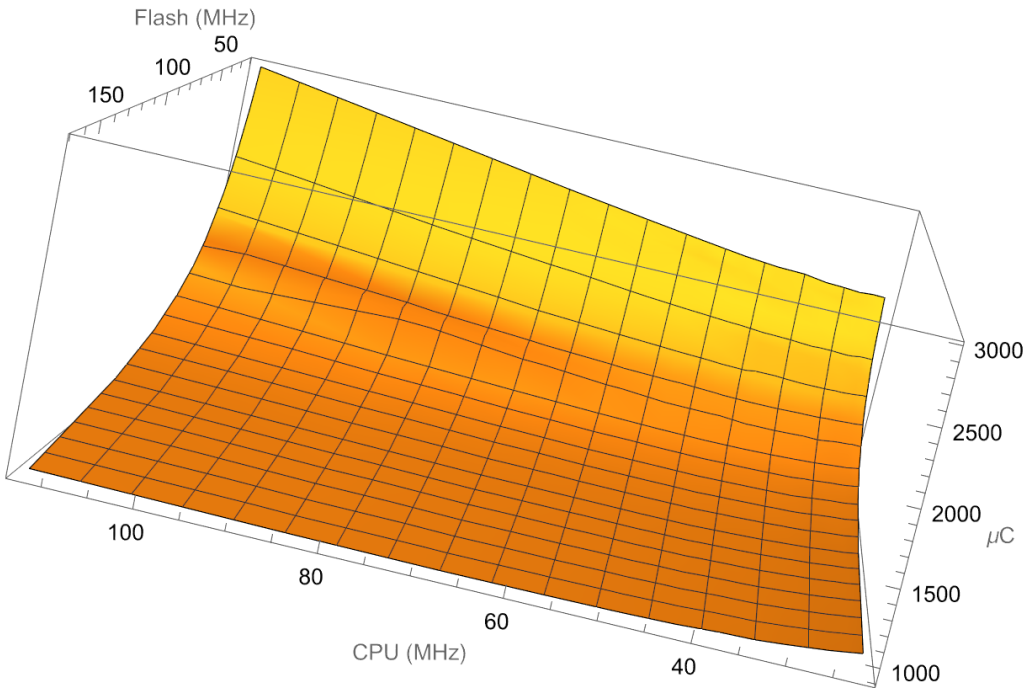

Figure 5.1: Electric Charge Consumption

Further more, focusing on Flash running at $168MHz$, the best frequency for CPU is $\sim 67MHz$, as shown in Figure 5.2. If Flash running at $192MHz$, the best frequency for CPU is $\sim 77MHz$. These two groups of numbers show that in the case of no cache, it's better to set the frequency of CPU to be 40% of that of Flash, proving that Flash is really the bottleneck.

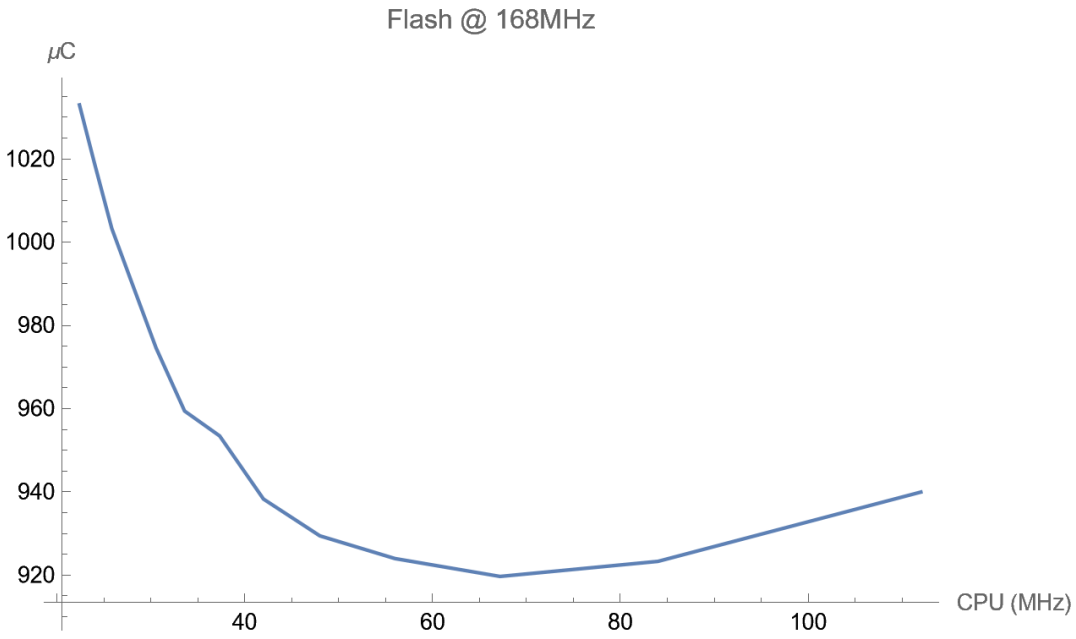

Figure 5.2: Electric Charge Consumption (Flash $168MHz$)

On the other hand, when cache becomes hot and has a high hit rate, it would be better to run

CPU as fast as possible and the impaction of the slower Flash can be neglected.

5.2.3 Computational Intensive vs Hardware Timing

There are computational intensive jobs, and there are slow hardware operations that need quite some time to complete. Different strategies shall be considered.

For computational intensive jobs, check section CPU on how to let it run more efficiently. For slow hardware operations, check if sleep modes can be utilized; if not, consider changing to a lower clock frequency for CPU, Flash, and notably *hclk*.

Chapter 6

FAQ

- **Q**: Why do the reconfiguration in event handler of waking up? The framework can remember all peripheral configurations and reconfigure them.
- **A**: Configure a peripheral is not an easy job like writing a bulk of registers. It's impractical to handle all peripherals one by one. On the other hand, it's easy for Apps to reconfigure them, as the code is ready there.

- **Q**: What happened to CPU in sleep modes of Category B?
- **A**: It's powered off.

- **Q**: How to debug programs when power saving is enabled?
- **A**: It depends on which part of programs need to be debugged.

  For the part between PLATFORM_CB_EVT_BEFORE_DEEP_SLEEP and PLATFORM_CB_EVT_ON_DEEP_SLEEP_WAKEUP, it's generally not debug-able.

  For other parts, firstly, disable power saving and debug it as usual. After everything is OK, re-enable power saving. Or, in the handler of PLATFORM_CB_EVT_QUERY_DEEP_SLEEP_ALLOWED check if debugger is attached, and if so, disallow sleep modes of Category B:

```
uint32_t query_deep_sleep_allowed(void *dummy,
  void *user_data)
{
  if (IS_DEBUGGER_ATTACHED())
    return 0;

  // ....
}
```